

# Web 安全与渗透测试入门指北

作者：

- Patricia of End
- HDdss
- zero6six
- linlinzzo（乱入

## 欢迎来到 Web 世界

现在，互联网已经成了我们生活中不可或缺的一部分。但你在访问网站的时候有没有想过，当你打开浏览器、输入网址并按下回车时，一个看似普通的网页背后，可能已经发生了几十次网络通信。

1. 寻找服务器：浏览器通过 DNS 将域名转换为服务器的 IP 地址。
2. 发送请求：浏览器与服务器进行 TCP 三次握手，之后按照 HTTP 或 HTTPS 协议向服务器请求资源。
3. 处理数据：后端程序执行业务逻辑，并根据需要查询数据库等。
4. 呈现页面：服务器返回 HTML、CSS 和 JavaScript，浏览器将它们解析成网页。

Web 安全所研究的，正是这些环节中可能出现的问题。不管是开发者写错的一行代码、还是不恰当的权限设置、又或者是未经检查的用户输入，甚至一个被遗忘的备份文件，都可能成为可被利用的致命的漏洞。

CTF Web 题目会把这些问题设计成安全、可控的挑战，引导你理解漏洞原理，并找到隐藏在题目环境中的 flag。

### 不需要担心自己的起点

如果你现在还不会编程、没用过 Linux，也不知道 HTTP 协议是什么——完全没关系。每一位 Web 选手，都曾经从第一次按下 **F12** 开始。

## 互联网与 Web 安全

### 1.1 什么是互联网？

互联网可以理解为一个由大量设备和网络连接起来的通信系统。我们的电脑、手机、服务器以及各种智能设备，都可以通过网络交换数据。

为了让不同设备能够理解彼此，它们需要遵守共同的通信规则，这些规则称为 **网络协议**。

| 协议   | 主要作用                     |
|------|--------------------------|
| IP   | 标识网络中的设备，并负责数据包的寻址与传递。   |
| TCP  | 建立可靠连接，尽可能确保数据完整、有序地到达。  |
| UDP  | 更加轻量、速度较快，但不保证数据可靠到达。    |
| DNS  | 将人类容易记忆的域名转换为 IP 地址。     |
| HTTP | 在浏览器、客户端与 Web 服务器之间传输数据。 |

| 协议        | 主要作用                   |
|-----------|------------------------|
| HTTPS     | 使用 TLS 加密保护通信过程的 HTTP。 |
| WebSocket | 在客户端和服务端之间建立持续的双向通信。   |

对 Web 方向而言，最需要优先掌握的协议之一就是 **HTTP** 协议。

## 1.2 什么是 Web?

Web 是建立在互联网之上的一种信息服务。我们平时访问的网站、浏览器中的管理后台，以及许多手机 APP 的服务端接口，本质上都可能属于 Web 应用。

- 前端通常运行在浏览器中，由 HTML、CSS 和 JavaScript 构成，负责页面结构、样式以及用户交互。
- 后端运行在服务器上，负责处理请求、验证身份与权限、查询数据库并向客户端返回响应。
- 数据库用于保存用户账号、文章、权限和配置等信息，常见的有 MySQL、PostgreSQL 和 SQLite。
- 浏览器与服务器通过 HTTP 请求和响应交换数据，它就像客户端与服务器之间往来的信件。

### 前端三件套

- **HTML**：描述页面的结构和内容。
- **CSS**：控制页面的颜色、布局和视觉样式。
- **JavaScript**：实现交互、数据请求和动态逻辑。

前端隐藏不等于安全，前端代码通常会被发送到用户的浏览器中，用户可以通过开发者工具查看和修改它。因此，密码、密钥和真正的权限校验不能只放在前端。

### 后端与数据库

常见的后端语言和框架包括：

PHP / Python Flask / Python Django / Java Spring / Node.js / Go / C# **ASP.NET**

常见数据库包括：

MySQL PostgreSQL SQLite Redis MongoDB

后端程序通常会根据用户输入构造数据库查询。如果程序信任并且直接拼接用户输入，同时没有正确使用参数化查询，就可能产生 **SQL 注入漏洞**。

### HTTP 请求与响应

浏览器向服务器发送的请求（Request）可能形如这样：

```
GET /index.html HTTP/1.1
Host: xdsec.org
User-Agent: Mozilla/5.0
Cookie: session=abcdef
```

服务器处理请求后，返回的响应（Response）可能形如这样：

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

<h1>Hello, Web!</h1>
```

常见 [HTTP 请求方法](#) 如下，关于每个方法并没有法定的用途，更像是一种约定，这里列出的常见用途基本遵循开发接口文档设计的 [RESTful API 最佳实践](#)。

| 方法      | 常见用途                                  |
|---------|---------------------------------------|
| GET     | 获取页面或数据。                              |
| POST    | 提交表单、创建数据或执行操作。                       |
| PUT     | 更新已有数据。                               |
| DELETE  | 删除指定数据。                               |
| OPTIONS | 查询服务器支持的通信方式。                         |
| HEAD    | 类似于 GET 请求，返回的响应中没有具体的内容，用于获取报头。      |
| PATCH   | 是对 PUT 方法的补充，用来对已知资源进行局部更新。           |
| TRACE   | 主要用于测试或诊断。                            |
| CONNECT | 要求在与代理服务器通信时建立隧道，实现用隧道协议进行TCP通信。极少使用。 |

常见 [HTTP 状态码](#) 如下，除了常见的状态码之外，你可以约定一些状态码，例如 [418 I'm a teapot](#)。

| 状态码          | 原因短语                         | 含义                                  |
|--------------|------------------------------|-------------------------------------|
| 200          | OK                           | 请求成功。                               |
| 301 /<br>302 | Moved Permanently /<br>Found | 资源被重定向到了其他位置。                       |
| 400          | Bad Request                  | 请求格式或参数存在错误。                        |
| 401          | Unauthorized                 | 用户尚未通过身份认证。                         |
| 403          | Forbidden                    | 服务器理解请求，但拒绝访问。                      |
| 404          | Not Found                    | 请求的资源不存在。                           |
| 500          | Internal Server Error        | 服务器内部发生错误。                          |
| 502          | Bad Gateway                  | 服务器作为网关需要得到一个处理这个请求的响应，但是得到一个错误的响应。 |

在 Web CTF 中，我们经常需要观察、修改并重新发送 HTTP 请求。因此，理解 HTTP 协议是学习 Web 安全的重要基础。

### 1.3 什么是 Web 安全？

Web 安全研究的是 Web 应用在设计、开发、部署和运行过程中可能出现的安全问题。漏洞可能来自很多不同的环节：

- 用户输入没有被正确检查和处理；
- 身份认证机制存在缺陷；
- 权限检查不完整；
- 敏感信息被暴露在前端；
- 文件上传功能缺少限制；
- 数据库查询语句被错误拼接；
- 模板引擎或解析器被不安全地使用；
- 使用了存在已知漏洞的第三方组件；
- 服务器配置错误；
- 开发过程中遗留了调试接口、源码或备份文件。

在 CTF 中，我们通常需要利用题目有意设置的漏洞，找到形如下面内容的通关凭证：

```
fakeflag{This_is_an_example_fakeflag}
```

现实中的渗透测试则必须建立在明确授权的基础上。测试人员模拟攻击者，发现并验证安全问题，最终向系统所有者提供报告和修复建议。

Web 安全的目标不是破坏网站，而是发现问题、理解问题、验证风险，并帮助开发者修复问题。

## 1.4 常见的 Web 安全问题

初学阶段不需要一次掌握所有漏洞。你可以先了解它们解决的是什么问题，再逐步学习漏洞产生、利用和修复的原理。你可以自己搭建一个网站，从网站日志中窥见真实网络中的网络攻击。

- **信息泄露**：HTML 注释、JavaScript、配置文件、备份文件、错误信息、Git 仓库文件或调试接口意外暴露了源代码或敏感数据。
- **SQL 注入**：程序将用户输入直接拼接到 SQL 语句中，导致攻击者能够改变原有的数据库查询逻辑。
- **跨站脚本攻击**：应用将未经安全处理的用户输入输出到页面，使浏览器将其作为 HTML 或 JavaScript 解析执行。
- **跨站请求伪造**：攻击者诱导已登录用户发送非预期请求，利用了浏览器可能自动携带 Cookie 的特性。
- **文件上传漏洞**：网站没有正确检查上传文件的类型、内容、路径、名称或访问方式，从而产生安全风险。
- **路径穿越与文件包含**：用户能够控制文件路径，使程序读取或包含本不应该被访问的文件。
- **服务端请求伪造**：攻击者操纵服务器向非预期地址发起请求，可能借此访问服务器所在的内部网络。
- **服务端模板注入**：用户输入被当作模板代码的一部分解析，导致模板引擎执行非预期表达式或操作。
- **反序列化漏洞**：程序将外部可控的数据还原成对象时，可能触发危险方法或非预期的程序逻辑。
- **认证与访问控制漏洞**：身份验证、Session、JWT 或权限检查存在缺陷，导致用户访问不属于自己的数据或功能。

## 1.5 Web 安全与渗透测试

一次规范的渗透测试通常包含以下阶段：

1. 确认测试目标、授权范围和时间；
2. 收集目标系统的信息；
3. 识别技术栈与暴露的攻击面；
4. 分析可能存在的漏洞；

5. 在可控范围内验证漏洞；
6. 评估漏洞的实际影响；
7. 清理测试过程中产生的数据；
8. 编写报告并提供修复建议。

我们就 CTF 和真实的渗透测试做一个对比：

| CTF           | 真实渗透测试           |
|---------------|------------------|
| 目标通常是获取 flag。 | 目标是发现、评估并降低安全风险。 |
| 题目一般确定存在解法。   | 目标系统未必存在可以利用的漏洞。 |
| 环境通常可以重置。     | 必须尽可能避免影响正常业务。   |
| 更强调知识点和解题技巧。  | 更强调授权、流程、证据和报告。  |
| 可以在题目规则内大胆尝试。 | 必须严格遵守测试范围和操作限制。 |

## 1.6 法律与授权边界

**==无授权，不测试；有授权，不越界==**

学习安全技术不代表可以随意测试互联网上的网站。未经授权的扫描、漏洞验证和数据访问都可能造成严重后果。也就是所谓的伸手必被捉

1. 只测试自己拥有或得到明确授权的系统；
2. 使用公开靶场前，先阅读并遵守平台规则；
3. 不要扫描或攻击学校、企业及公共机构的网站；
4. 不要因为“只是试一下”就向陌生网站发送攻击请求；
5. 不要获取、保存或传播他人的隐私和业务数据；
6. 发现真实漏洞时，应通过正规漏洞响应渠道报告；
7. 不要在生产环境中运行来源不明的攻击脚本。

我们强烈推荐在以下环境中练习：

- 自己控制的虚拟机或服务器；
- 本地 Docker 容器；
- 自己编写的测试程序；
- 比赛提供的题目环境；
- 明确允许安全测试的公开靶场。

## 如何学习 Web 安全？

### 2.1 先理解，再使用工具

各种各样的 Web 安全工具能够提高效率，但工具不能替代你对原理的理解。代理工具可以帮助我们修改 HTTP 请求，但如果不知道请求头、Cookie 和请求体分别有什么作用，就很难理解修改请求后为什么会产生不同结果。

推荐采用下面的学习顺序：

1. 理解基本概念；
2. 手动完成一次实验；
3. 使用工具重复实验；
4. 阅读并理解工具产生的结果；
5. 尝试编写简单脚本实现相同功能；
6. 总结漏洞产生和修复的原因。

不要一开始就陷入“工具收藏”。会使用几十个工具，不如真正理解一次 HTTP 请求。

## 2.2 搜索、AI 与提问

### 善用搜索引擎

学习 Web 安全时，你会经常遇到错误信息、陌生术语和环境问题。搜索能力是非常重要的基础能力。

搜索时，可以提供以下信息：

- 完整的错误信息；
- 使用的软件及其版本；
- 操作系统和运行环境；
- 编程语言或框架；
- 希望实现的目标；
- 已经尝试过的方法。

不建议只搜索

PHP 报错怎么办？？？

可以搜索更加明确的内容：

PHP 8.3 undefined array key POST form

也可以直接搜索完整报错：

"Warning: Undefined array key" PHP POST

我们推荐你优先参考：

1. 官方文档；
2. OWASP、MDN 等可信技术资料；
3. 项目的官方 GitHub 仓库和 Issue；
4. 高质量技术社区和博客；
5. 经过交叉验证的其他搜索结果。

### 正确使用 AI

不可不提的是，现如今生成式人工智能模型对所有 CTF 赛事的冲击都很大。许多的赛事已经变成了“宝可梦对战赛”，Agent 能力强，烧的 token 多，就可以更多、更快的解出题目。但是这样的“成功”无疑是比大模型幻觉更

加严重的空中楼阁，在 AI 的时代静下心来打好基础，逐步提升自己，才是抓住时代浪潮，从其他的“AI 小子”中脱颖而出的关键。AI 并非洪水猛兽，且可以成为你学习路上最好的良师益友。

AI 可以辅助你：

- 解释陌生概念；
- 阅读错误信息；
- 分析代码执行流程；
- 生成用于学习的简单示例；
- 比较不同技术方案；
- 整理笔记和排查问题。

我们并不建议你吧题目直接丢给 AI（所谓的一把梭）。这是一个面向新人的赛事，解出题目不是关键，能学到知识才是我们最希望你能收获的事情。

向 AI 提问时，最好提供：

- 题目或实验环境；
- 相关代码；
- HTTP 请求和响应；
- 使用的软件版本；
- 已经尝试过的操作；
- 实际结果与预期结果；
- 自己不理解的具体位置。

AI 也可能出错

对关键结论应查阅官方文档并在本地验证。不要直接运行自己不了解的命令，也不要将真实密码、Cookie、Token 或私有代码发送给不可信服务。

## 学会如何提问

可以阅读平台上的[《提问的艺术》](#)。

一个高质量的问题通常包含：

1. 我正在做什么？
2. 环境和软件版本是什么？
3. 我预期发生什么？
4. 实际上发生了什么？
5. 我已经尝试了哪些方法？
6. 我具体不理解哪一步？
7. 是否附上了代码、请求、响应和报错？

不推荐这样提问：

师傅，这题怎么做？我怎么什么都不会？

推荐这样描述问题：

我猜测这道题可能存在 SQL 注入，因为输入单引号后页面返回了数据库错误。我已经尝试判断列数，并使用 Burp Suite 重放了请求。下面是原始请求、响应和我测试过的输入。目前的问题是两次响应看起来完全一样，我不知道应该比较响应长度还是响应时间。请问我的判断方向是否正确？

2.3 掌握必要的基础知识

- Linux 与计算机基础：学习文件、目录、权限、进程、端口、环境变量以及基础 Shell 命令。
- HTTP 与浏览器：理解 URL、请求方法、请求头、Cookie、Session、状态码和重定向。
- HTML、CSS 与 JavaScript：能看懂页面结构、表单、DOM 操作、事件处理、Fetch 和 JSON。
- 一门后端语言：可以从 PHP 或 Python 开始，学习参数接收、文件操作、数据库和 Session。
- 数据库与 SQL：掌握增删改查、WHERE、UNION、参数化查询和数据库用户权限。
- 编码与数据格式：认识 Base64、URL 编码、HTML 实体、十六进制、JSON、XML 和哈希。
- Linux 基础命令：你不需要一次背下所有命令。先理解每条命令解决了什么问题，在真正需要使用时再查询文档。

你可以使用 [VMWare](#) 虚拟机或者 [WSL \(Windows Subsystem for Linux\)](#) 来使用 Linux 操作系统。我们推荐你使用 [Kali Linux](#) 这个 Linux 发行版来进行解题辅助和渗透测试。

这些其实也是“开发与运维基础”方向需要的知识。看到这么多东西无从下手？去做做运维题吧！（狗头保命）

浏览器开发者工具

你可以按下 **F12** 或 **Ctrl + Shift + I** 打开开发者工具。

| 面板               | 主要用途                                       |
|------------------|--------------------------------------------|
| Elements / 元素    | 查看并临时修改 HTML 和 CSS。                        |
| Console / 控制台    | 运行 JavaScript、查看日志和错误信息。                   |
| Sources / 源代码    | 查看和调试网页加载的脚本及其他资源。                         |
| Network / 网络     | 查看 HTTP 请求、响应、参数和资源加载过程。                   |
| Application / 应用 | 查看 Cookie、LocalStorage、SessionStorage 等数据。 |

编码不等于加密

Web 题目中经常出现：

Base64 URL Encode HTML Entity Hex Unicode JSON XML MD5 SHA-256 JWT

请区分三个概念：编码用于表示数据，加密用于保护数据机密性，哈希通常用于产生固定长度摘要。Base64 只是一种编码，并不能保护秘密。

2.4 推荐学习路线

1. 认识 Web：学习 HTML 和 HTTP，使用开发者工具查看页面结构、请求参数和 Cookie。
2. 搭建第一个网站：使用 PHP 或 Flask 在本地创建表单、接收参数并连接数据库，[推荐查看运维方向的入门指北](#)。
3. 学习基础漏洞：优先学习信息泄露、SQL 注入、XSS、路径问题和文件上传。
4. 学习认证与服务端漏洞：继续了解 Cookie、Session、JWT、CSRF、SSRF、SSTI、XXE 和反序列化。



5. 综合练习与复盘：完成靶场题目，编写简单自动化脚本，并用自己的语言总结漏洞原理。

每学习一个漏洞，可以按照下面的步骤进行：

1. 编写一段存在漏洞的代码；
2. 在本地环境中复现漏洞；
3. 使用代理工具观察请求和响应；
4. 修改代码并修复漏洞；
5. 比较修复前后的行为；
6. 找一道同类型靶场题进行练习。

比起记住大量 payload，更重要的是理解：数据从哪里进入，经过哪些处理，最终到达了什么位置。

## 2.5 常用工具

### 浏览器开发者工具

查看 HTML、JavaScript、HTTP 请求、Cookie 和前端代码，是最容易获得、也最应该先学会的工具。

### Burp Suite

付费渗透工具，可以拦截、修改和重放 HTTP 请求。初期可以先学习 Proxy、HTTP History、Repeater 和 Decoder。

### Yakit

国产的免费渗透工具，推荐使用，与 Burp Suite 功能大致相同，安装更方便。

### curl

通过命令行发送网络请求，适合观察请求过程、测试接口和编写自动化流程。

获取一个网页：`\ncurl https://example.com/`

只查看响应头：`\ncurl -I https://example.com/`

发送 POST 请求：`\ncurl -X POST -d "username=guest&password=123456" https://example.com/login`

### CyberChef

用于处理 Base64、URL 编码、十六进制、哈希、JSON 和多层编码。

### 虚拟机

VMware 或 VirtualBox 可以提供隔离的实验系统，适合搭建 Kali Linux 等环境。

### Docker

快速运行和销毁本地靶场，减少不同实验环境之间的依赖冲突。

## 2.6 练习、WriteUp 与笔记

可供选择的练习资源包括：

- [PortSwigger Web Security Academy](#)；
- [OWASP WebGoat](#)；
- [OWASP Juice Shop](#)；
- [DVWA](#)；
- [Upload-Labs](#)；
- [SQLi-Labs](#)；
- [CTFHub](#)；
- [攻防世界](#)；
- [BUUCTF](#)；
- [NSSCTF](#)；
- [BugKu](#)。

推荐采用“学习一个知识点，马上完成一到三道相关题目”的方式。如果卡住，可以查看提示或 WriteUp，但不要只复制最终 payload。

阅读 WriteUp 后，请尝试回答下面的问题：

1. 漏洞为什么会出现？
2. 用户输入经过了哪些处理？
3. payload 中每一部分有什么作用？
4. 为什么其他尝试没有成功？
5. 开发者应该如何修复？
6. 换一个环境后，这种方法是否仍然有效？

只收藏 WriteUp 很容易产生“我好像会了”的错觉。**用自己的语言复述原理，才是真正理解的开始。**

一个简单的四周计划

| 时间  | 学习内容                                | 阶段目标                       |
|-----|-------------------------------------|----------------------------|
| 第一周 | HTML、浏览器开发者工具、HTTP、GET、POST、Cookie  | 理解网页和请求的基本结构，找到本文隐藏的 flag。 |
| 第二周 | PHP 或 Flask、SQLite 或 MySQL、表单与参数处理  | 搭建一个可以接收输入的简单动态网站。         |
| 第三周 | 信息泄露、SQL 注入、XSS、路径穿越、文件上传           | 每类漏洞至少完成一次本地复现。            |
| 第四周 | Burp Suite、CTF 入门题、Python 请求脚本、复盘笔记 | 独立完成数道基础 Web 题并总结原理。       |

学习速度并不重要。保持动手、持续思考，并且真正弄懂每一次请求，才是最有效的进步方式。

从第一个 flag 开始

Web 安全并不是由一串神秘 payload 组成的魔法。

它建立在许多基础知识之上：

- 浏览器如何解析页面；
- HTTP 如何传输数据；
- 后端如何处理用户输入；
- 数据库如何执行查询；
- 身份认证和权限控制如何实现；
- 开发者为什么会写出存在安全问题的代码。

刚开始学习时，你可能会因为环境配置失败而烦躁，会看不懂请求和响应，也可能对着一道题几个小时没有思路。

这些都是正常的。不要只追求“做出了多少题”，也不要因为暂时看不懂 WriteUp 而怀疑自己。

每弄懂一个请求头、每读懂一段代码、每独立定位一次错误，都是实实在在的进步。

**欢迎来到 MoeCTF 2026 Web 方向**

愿你保持好奇、尊重边界，享受思考与探索的过程。